

Fundación Fulgor
PROCOM

Informe

Plataforma de Verificación

Autor: Alfici, Facundo Ezequiel

Documento: 44012327

Docentes: Ariel Pola, Santiago Garaventta Pascual y Agustín Pesce

31 de agosto de 2026

Índice

1. Introducción	3
2. Desarrollo	4
2.1. Firmware del MicroBlaze	4
2.2. Programa en Python	5
2.3. Ejemplos de uso	5
3. Conclusión	8

1. Introducción

En este trabajo se realizó un programa en lenguaje C, con el que se programa el MicroBlaze que se utilizó en el trabajo, y un programa en Python que se encarga de comunicarse con la placa por medio de la librería PySerial. Los desafíos principales se muestran a continuación.

- Controlar el led a encender o apagar (0, 1, 2 o 3). El color de cada led puede ser rojo, azul, verde o cualquier combinación de ellos
- Leer el estado de los switches de la placa, manejados por medio de las Virtual Input/Output
- Definir una trama para el envío de los datos, considerando Device y Data como se deseen.
- Ejemplo de desarrollo en la siguiente figura.

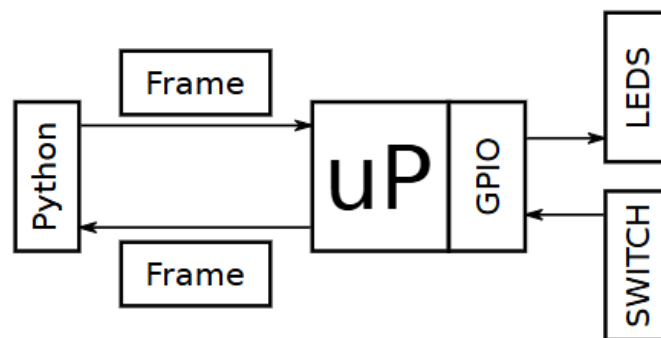


Figura 1.0.1: Diagrama en bloques del flujo completo.

2. Desarrollo

2.1. Firmware del MicroBlaze

En este inciso se detallará la estructura del código que conforma el firmware cargado a la FPGA, encargado de manejar el **MicroBlaze**. Escrito en lenguaje C, es el encargado de controlar las acciones que permiten la lectura y escritura de las entradas y salidas de propósito general de la FPGA.

Utilizando las librerías `xgpio.h` y `xuartlite.h` se puede controlar los elementos necesarios para realizar el trabajo solicitado. En primera medida, se definieron los devices a leer/escribir, siendo los 4 leds y los switches de la placa. A los leds se les cargarán ciertos valores según el color a mostrar, permitiendo controlar el RGB de cada led a voluntad. Algo importante a tener en cuenta fue la selección de la trama, la cual fue dejada a consciencia del desarrollador, por ello, se eligió la trama mostrada por el profesor, la cual se muestra en la siguiente figura.

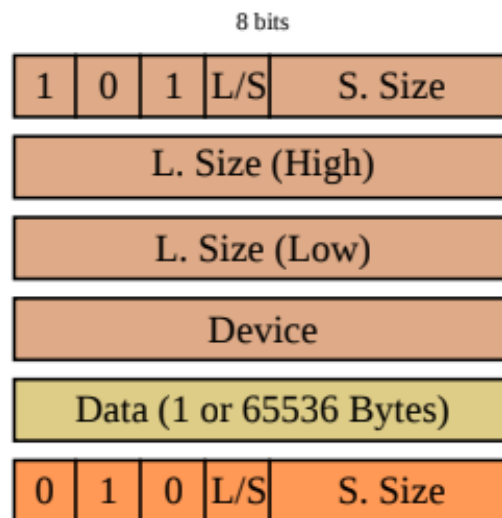


Figura 2.1.1: Construcción de la trama de envío y recepción.

Con esto, el programa se encargará de enviar y recibir tramas definidas por un **Header**, compuesto de una combinación de bits para indicar el inicio, tipo de trama, cantidad de bits de la información y el dispositivo a utilizar. Luego se envía la **Data** y se cierra el ciclo con otra combinación de bits llamada **Trailer o Fin de trama** que contendrá una combinación de cierre, el tipo de trama y la cantidad de bits en caso de ser corta.

Con esto, se colocaron diferentes verificaciones dentro del código, de tal manera que si hay un corrimiento de trama debido a algun error del hardware o de python

la trama se descarte, lo que permite mayor robustez en el comportamiento del micro.

2.2. Programa en Python

En lo que respecta a la estructura del programa realizado en Python, se utilizó la librería `PySerial` para poder comunicarse con la placa por medio del puerto serial. La configuración se realizó a una velocidad de 115200 baudios, sin bit de paridad, 8 bits de datos y 1 bit de stop. Se definieron las funciones necesarias para poder enviar y recibir tramas, las cuales se encargan de construir la trama según lo definido en el firmware del microblaze. Estos comandos se detallan a continuación:

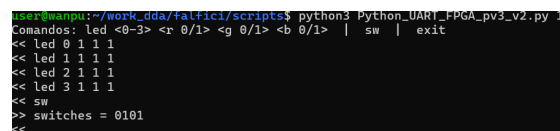
- Comando de encendido de led `"led <0-3><R 0/1><G 0/1><B 0/1>"`: Permite encender los 3 colores posibles de cada led.
- Comando de lectura de switches `"sw"`: Permite leer el estado de los switches de la placa.
- Comando de salida `".exit"`: Permite salir del programa.

Dado que el intercambio de tramas entre el microblaze y el programa en python es bidireccional, se definieron funciones similares en ambos extremos para poder parsear la trama recibida y verificar que sea correcta, de lo contrario se descarta.

2.3. Ejemplos de uso

Algunos ejemplos de uso se mostrarán en este inciso, donde se podrá ver el funcionamiento del sistema completo. Para poder programar el MicroBlaze se utilizó el programa **Vitis 2024.2**, el cual permite compilar, debuggear y cargar remotamente el firmware a una placa específica seleccionada previamente al activar el tunel a través de la terminal. Una vez cargado el firmware, se puede ejecutar el programa en Python dentro de la misma terminal, de tal manera que comienza la comunicación entre la FPGA y la consola de Python.

Se muestran a continuación el uso de todos los comandos y la respuesta obtenida observada en las entradas del VIO.



```
user@manpu:~/work_dda/fal/fici/scripts$ python3 Python_UART_FPGA_pv3_v2.py -
Comandos: led <0-3> <r 0/1> <g 0/1> <b 0/1> | sw | exit
<< led 0 1 1 1
<< led 1 1 1 1
<< led 2 1 1 1
<< led 3 1 1 1
<< sw
>> switches = 0101
<<
```

Figura 2.3.1: Ejmplo N^o1 de uso de los comandos definidos.

Donde se puede ver que se activan los 4 leds, en todos los colores posibles y se lee el valor del switch de la placa.

u_violprobe_in0_0_1[2:0]	[H] 7	Input	hw_vio_1
u_violprobe_in0_0_1[2]	●	Input	hw_vio_1
u_violprobe_in0_0_1[1]	●	Input	hw_vio_1
u_violprobe_in0_0_1[0]	●	Input	hw_vio_1
u_violprobe_in1_0_1[2:0]	[H] 7	Input	hw_vio_1
u_violprobe_in1_0_1[2]	●	Input	hw_vio_1
u_violprobe_in1_0_1[1]	●	Input	hw_vio_1
u_violprobe_in1_0_1[0]	●	Input	hw_vio_1
u_violprobe_in2_0_1[2:0]	[H] 7	Input	hw_vio_1
u_violprobe_in2_0_1[2]	●	Input	hw_vio_1
u_violprobe_in2_0_1[1]	●	Input	hw_vio_1
u_violprobe_in2_0_1[0]	●	Input	hw_vio_1
u_violprobe_in3_0_1[2:0]	[H] 7	Input	hw_vio_1
u_violprobe_in3_0_1[2]	●	Input	hw_vio_1
u_violprobe_in3_0_1[1]	●	Input	hw_vio_1
u_violprobe_in3_0_1[0]	●	Input	hw_vio_1
u_violvio_0_probe_out0	[B] 0	Output	hw_vio_1
u_violvio_0_probe_out1[3:0]	[B] 0'101	Output	hw_vio_1

Figura 2.3.2: Salida obtenida en VIO.

```
<< led 0 0 0 0
<< led 1 0 0 0
<< led 3 0 0 0
<< sw
>> switches = 1111
<< |
```

Figura 2.3.3: Ejmplo N^o2 de uso de los comandos definidos.

Donde se puede ver que se apagan los leds 0, 1 y 3, dejando el 2 encendido en su estado anterior, es decir, RGB activados. Nuevamente se lee el valor de los switches, pero en este caso se realizó un cambio mediante el VIO.

Name	Value	Acti...	Directi...	VIO
u_violprobe_in0_0_1[2:0]	[H] 0		Input	hw_vio_1
u_violprobe_in0_0_1[2]	●		Input	hw_vio_1
u_violprobe_in0_0_1[1]	●		Input	hw_vio_1
u_violprobe_in0_0_1[0]	●		Input	hw_vio_1
u_violprobe_in1_0_1[2:0]	[H] 0		Input	hw_vio_1
u_violprobe_in1_0_1[2]	●		Input	hw_vio_1
u_violprobe_in1_0_1[1]	●		Input	hw_vio_1
u_violprobe_in1_0_1[0]	●		Input	hw_vio_1
u_violprobe_in2_0_1[2:0]	[H] 7		Input	hw_vio_1
u_violprobe_in2_0_1[2]	●		Input	hw_vio_1
u_violprobe_in2_0_1[1]	●		Input	hw_vio_1
u_violprobe_in2_0_1[0]	●		Input	hw_vio_1
u_violprobe_in3_0_1[2:0]	[H] 0		Input	hw_vio_1
u_violprobe_in3_0_1[2]	●		Input	hw_vio_1
u_violprobe_in3_0_1[1]	●		Input	hw_vio_1
u_violprobe_in3_0_1[0]	●		Input	hw_vio_1
u_violvio_0_probe_out0	[B] 0		Output	hw_vio_1
u_violvio_0_probe_out1[3:0]	[B] 1111		Output	hw_vio_1

Figura 2.3.4: Salida obtenida en VIO.

```
>> switches = 1111
<< led 2 0 0 0
<< sw
>> switches = 0000
<< |
```

Figura 2.3.5: Ejmplo N^o3 de uso de los comandos definidos.

Finalmente, se apaga el ultimo led encendido y se deja el estado de los switches en 0.

u_violprobe_in0_0_1[2:0]	[H] 0		Input	hw_vio_1
u_violprobe_in0_0_1[2]	●		Input	hw_vio_1
u_violprobe_in0_0_1[1]	●		Input	hw_vio_1
u_violprobe_in0_0_1[0]	●		Input	hw_vio_1
u_violprobe_in1_0_1[2:0]	[H] 0		Input	hw_vio_1
u_violprobe_in1_0_1[2]	●		Input	hw_vio_1
u_violprobe_in1_0_1[1]	●		Input	hw_vio_1
u_violprobe_in1_0_1[0]	●		Input	hw_vio_1
u_violprobe_in2_0_1[2:0]	[H] 0		Input	hw_vio_1
u_violprobe_in2_0_1[2]	●		Input	hw_vio_1
u_violprobe_in2_0_1[1]	●		Input	hw_vio_1
u_violprobe_in2_0_1[0]	●		Input	hw_vio_1
u_violprobe_in3_0_1[2:0]	[H] 0		Input	hw_vio_1
u_violprobe_in3_0_1[2]	●		Input	hw_vio_1
u_violprobe_in3_0_1[1]	●		Input	hw_vio_1
u_violprobe_in3_0_1[0]	●		Input	hw_vio_1
u_violvio_0_probe_out0	[B] 0		Output	hw_vio_1
u_violvio_0_probe_out1[3:0]	[B] 0000		Output	hw_vio_1

Figura 2.3.6: Salida obtenida en VIO.

3. Conclusión

Como conclusión, se cierra este informe destacando la dificultad inicial de manejar el entorno de **Vitis** y la consola **XSDB**, aunque no se tuvieron mayores problemas que ese. El programa envía y recibe las tramas correctamente, lo que permitirá futuras implementaciones de control de la placa mediante el puerto serial de Python y el **Protocolo UART** de la FPGA. Se destaca la **robustez** del sistema ante cualquier error de comunicación, ya que si se pierde la sincronización de la trama, el MicroBlaze descarta la trama y espera la siguiente, evitando errores en el control de los leds y switches.